

BACSE202 – Database Systems

B-Tree and B⁺-Tree Indexing – Complete Notes

Module 3 (dynamic multilevel indexing). Built from the course slides (BACSE202-DBS-Module-3, slides 136–160; Types_of_Indexes_in_DBMS, slides 38–61) and Elmasri & Navathe Ch. 17.

Contents

1 Why B-Trees? (Dynamic Multilevel Indexing)	1
2 B-Tree	2
2.1 Definition (order p)	2
2.2 B-tree insertion	2
2.3 B-tree deletion (summary)	2
2.4 B-tree index slides (as given in the course)	2
3 B⁺-Tree	3
3.1 Properties	3
3.2 Node structure	3
3.3 Order and occupancy rules	3
3.4 Search	4
4 B⁺-Tree Insertion	4
5 B⁺-Tree Deletion	6
5.1 Deletion cases (slides, order $m = 3$: max 2 keys, min 1 key per non-root node)	6
5.2 Deletion example from the construction problem (order $m = 4$; delete 6, 8, 10, 12)	7
5.3 Delete 52 (slide – result not shown there)	7
6 B-Tree vs. B⁺-Tree	7
7 Index Definition in SQL	8
8 Slide Errata	8
9 Quick Revision	8

1 Why B-Trees? (Dynamic Multilevel Indexing)

- A **static multilevel index** (e.g. ISAM) stores every level as a *sorted file*. Inserts and deletes shift entries at every level, so **overflow blocks** pile up, **performance degrades** as the file grows, and the **whole file must be reorganised** periodically.
- **Dynamic multilevel index**: each level is made of **tree nodes (one node = one disk block)** with **free space** left in every node. Inserts and deletes are handled by **local split / merge** of nodes, so the tree stays **balanced** automatically.
- The two such structures are the **B-tree** and its variant the **B⁺-tree**. B⁺-trees are used in almost every DBMS.

Disadvantage of indexed-sequential (ISAM) files	Advantage of B ⁺ -tree index files
Performance degrades as the file grows (many overflow blocks).	Automatically reorganises itself with small, local changes on insert/delete.
Periodic reorganisation of the entire file is required.	Reorganisation of the entire file is not required.

Minor disadvantage of B⁺-trees: extra insert/delete overhead and space overhead (nodes may be half empty). The advantages outweigh it.

B-Tree

2.1 Definition (order p)

A B-tree node has the form

$$\langle P_1, \langle K_1, Pr_1 \rangle, P_2, \langle K_2, Pr_2 \rangle, \dots, \langle K_{q-1}, Pr_{q-1} \rangle, P_q \rangle, \quad q \leq p$$

where P_i = **tree pointer** (to a child) and Pr_i = **data pointer** (to the record with key K_i).

B-tree rules

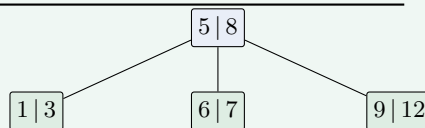
1. Keys in a node are sorted: $K_1 < K_2 < \dots < K_{q-1}$. Every key X in subtree P_i satisfies $K_{i-1} < X < K_i$.
2. Each node has **at most** p tree pointers (so at most $p - 1$ keys).
3. Each node **except the root and the leaves** has **at least** $\lceil p/2 \rceil$ tree pointers.
4. The root has at least 2 tree pointers, unless it is the only node.
5. A node with q tree pointers has $q - 1$ keys.
6. **All leaves are at the same level** (balanced). Leaf tree pointers are null.
7. **Every node stores data pointers, and each key appears exactly once** in the tree.

2.2 B-tree insertion

1. Search down to the correct **leaf** and insert the key in sorted order.
2. If the node now has p keys (overflow), **split** it: the **median key moves up** to the parent (it is *not* kept below). The left and right halves become two nodes.
3. If the parent overflows, split it too. If the root splits, a new root is created and the tree grows **one level taller at the top**.

B-tree of order $p = 3$ (max 2 keys/node): insert 8, 5, 1, 7, 3, 12, 9, 6

Insert	Action	Tree after
8, 5	–	[5 8]
1	[1 5 8] overflow → median 5 moves up	[5] → [1], [8]
7	7 > 5	[5] → [1], [7 8]
3	3 < 5	[5] → [1 3], [7 8]
12	[7 8 12] overflow → median 8 moves up	[5 8] → [1 3], [7], [12]
9	9 > 8	[5 8] → [1 3], [7], [9 12]
6	5 < 6 < 8	[5 8] → [1 3], [6 7], [9 12]



Final B-tree: 2 levels; every key appears once and carries its own data pointer.

2.3 B-tree deletion (summary)

- **Key in a leaf**, leaf stays at least half full ⇒ just remove it.
- **Key in an internal node** ⇒ replace it with its **in-order predecessor or successor** (always in a leaf), then delete that from the leaf.
- **Underflow** (fewer than $\lceil p/2 \rceil - 1$ keys) ⇒ **borrow** from a sibling through the parent (rotation), or **merge** with a sibling *plus the separating key pulled down from the parent*. This may propagate upward, and if the root becomes empty, height -1 .

2.4 B-tree index slides (as given in the course)

- B-tree index is the widely used data structure for **tree-based indexing**. It is a **balanced multilevel** search tree.
- All leaf nodes hold actual data pointers, and the leaves are interlinked by a linked list, supporting **random and sequential access**. (Strictly, linked leaves are a B⁺-tree feature.)
- **Internal nodes**: at least $\lceil n/2 \rceil$ pointers (except the root), at most n pointers.
- **Leaf nodes**: at least $\lceil n/2 \rceil$ record pointers and keys; at most n . Every leaf has one block pointer P to the next leaf.

3 B⁺-Tree

3.1 Properties

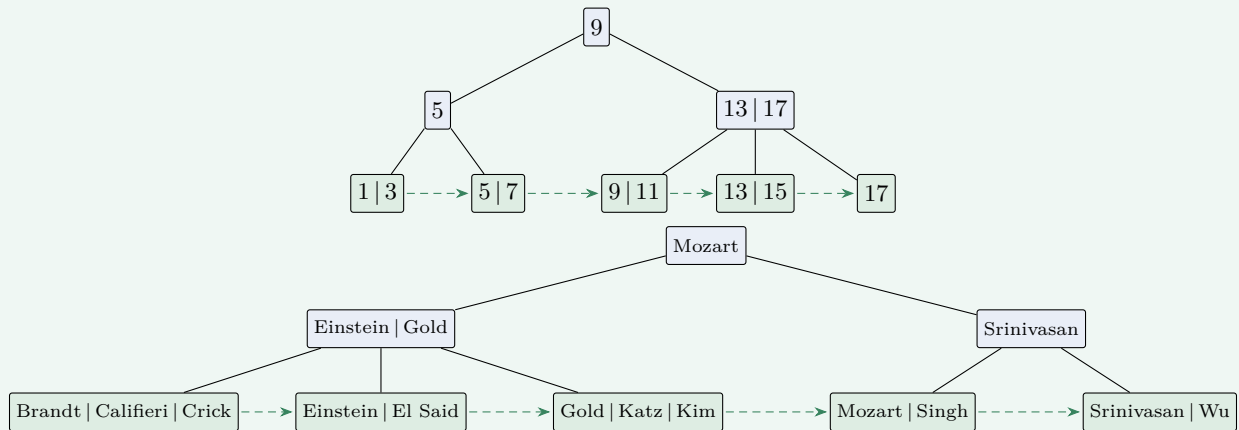
- A **balanced multiway** search tree used as a dynamic multilevel index. The slides say “balanced binary search tree”; it is really m -ary.
- **All leaves at the same height**: every root-to-leaf path has the same length.
- **Internal nodes hold only keys + child pointers**. They are just “signposts” (a sparse index over the leaves).
- **Leaves hold every key**, each with a pointer to its record (or bucket of records).
- **Leaves are linked** in key order (next-leaf pointer), which supports **random** access (root→leaf) and **sequential/range** access (leaf→leaf).

3.2 Node structure

$$\boxed{P_1 \mid K_1 \mid P_2 \mid K_2 \mid \cdots \mid P_{n-1} \mid K_{n-1} \mid P_n} \quad K_1 < K_2 < \cdots < K_{n-1}$$

- **Internal node**: P_i points to the subtree containing keys x with $K_{i-1} \leq x < K_i$.
- **Leaf node**: P_i ($i = 1 \dots n - 1$) points to the record with key K_i ; P_n points to the **next leaf**.
- If L_i, L_j are leaves with $i < j$, every key of L_i is $\leq$ every key of L_j .

B⁺-tree examples from the slides



B⁺-tree on `instructor.name`; each leaf key points to its instructor record (e.g. Brandt → 83821, Comp. Sci., 92000).

3.3 Order and occupancy rules

Order m = maximum number of **child pointers** in a node. The slides give:

Property	Root	Internal (non-root)	Leaf
Maximum children	m	m	–
Minimum children	2 (if internal) or 1 (if leaf)	$\lceil m/2 \rceil$	–
Maximum keys	$m - 1$	$m - 1$	m (see note)
Minimum keys	1 (if internal) or 0 (if leaf)	$\lceil m/2 \rceil - 1$	$\lceil m/2 \rceil$ (see note)

Convention used in ALL worked examples (use this in the exam and state it)

$$\begin{aligned} \text{max keys (every node)} &= m - 1, & \text{min children (internal)} &= \lceil \frac{m}{2} \rceil, \\ \text{min keys (internal)} &= \lceil \frac{m}{2} \rceil - 1, & \text{min keys (leaf)} &= \lceil \frac{m-1}{2} \rceil \end{aligned}$$

The slide table says leaf max = m , but every slide example overflows a leaf at m keys (order 3 at 3 keys, order 4 at 4 keys). So in practice leaf max = $m - 1$. The construction-problem slide itself states leaf min = $\lceil (m - 1)/2 \rceil$.

	$m = 3$	$m = 4$
Max children	3	4
Max keys (every node)	2	3
Min children (non-root internal)	2	2
Min keys (non-root internal)	1	1
Min keys (leaf)	1	2
Root	≥ 2 children (unless it is a leaf)	
	same	

3.4 Search

1. Start at the root. In each internal node, follow the pointer to the left of the first key $> x$ (or the last pointer if $x \geq$ every key).
2. At the leaf, look for x .
3. **Range query** $[a, b]$: find the leaf for a , then follow the leaf links until a key $> b$.

$$\text{Cost} = \text{height} \approx \left\lceil \log_{\lceil m/2 \rceil} N \right\rceil \quad (\text{worst case}), \quad (+1 \text{ to read the data block})$$

E.g. $N = 10^6$, $m = 100$: $\lceil \log_{50} 10^6 \rceil = \lceil 3.53 \rceil = 4$ levels (usually 3), against ≈ 17 for binary search on a sorted file of 100,000 blocks.

4 B⁺-Tree Insertion

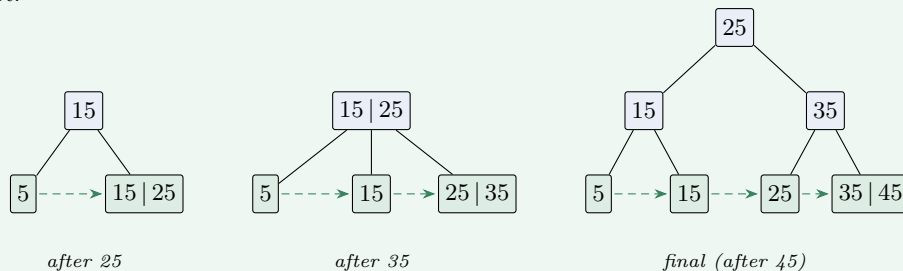
Algorithm

1. Every key is inserted into a **leaf**: search for the correct leaf N .
 2. Insert the key in sorted order.
 - **Case I** – leaf not full $\Rightarrow$ done.
 - **Case II** – leaf full $\Rightarrow$ **overflow**. Transfer a key to a sibling with room, **or split**:
 3. **Leaf split**: left leaf keeps $\lfloor m/2 \rfloor$ keys, the right gets the rest. **Copy** the first key of the right leaf **up** into the parent; it also stays in the leaf.
 4. **Internal split**: the middle key **moves up** (push up, not kept in either half).
 5. If the parent overflows, repeat. If the **root splits**, a new root is created and **height** +1.
- Leaf split** $\rightarrow$ **COPY up**. **Internal split** $\rightarrow$ **PUSH up**.

Example 1 (slides): order $m = 3$, insert 5, 15, 25, 35, 45

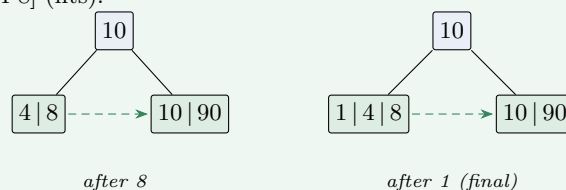
Max keys 2, min internal keys 1, leaf capacity 2.

- Insert 5, 15: [5 15].
- Insert 25: [5 15 25] overflow $\rightarrow$ [5] | [15 25], **copy 15 up**.
- Insert 35: leaf [15 25 35] overflow $\rightarrow$ [15] | [25 35], copy 25 up $\rightarrow$ root [15 25].
- Insert 45: leaf [25 35 45] overflow $\rightarrow$ [25] | [35 45], copy 35 up $\rightarrow$ root [15 25 35] overflow $\rightarrow$ **internal split, push 25 up** $\rightarrow$ new root.



Example 2 (slides): order $m = 4$, insert 10, 4, 90, 8, 1

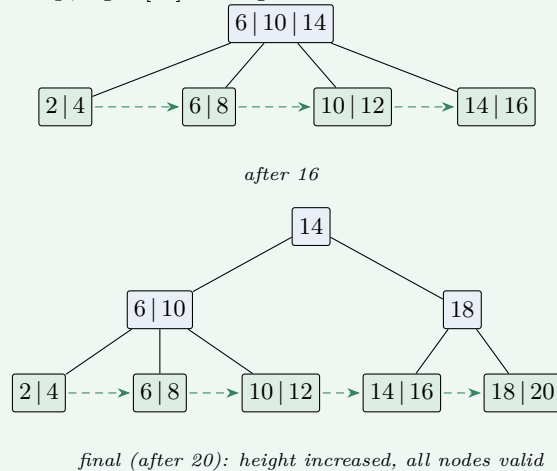
Max keys 3. Insert 10, 4, 90 $\rightarrow$ [4 10 90] (full). Insert 8 $\rightarrow$ [4 8 10 90] overflow $\rightarrow$ split [4 8] | [10 90], copy 10 up. Insert 1 $\rightarrow$ 1 < 10 $\rightarrow$ left leaf [1 4 8] (fits).



Example 3 (slide problem): order $m = 4$, insert 2, 4, 6, 8, 10, 12, 14, 16, 18, 20

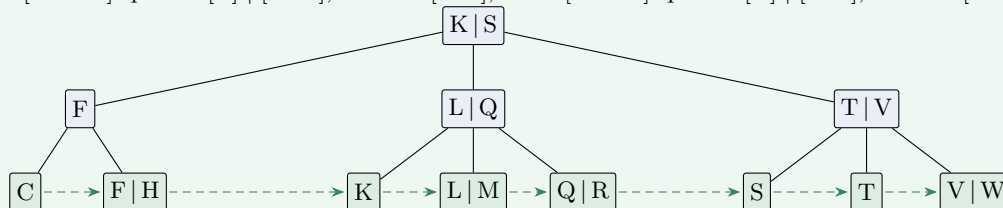
Max keys 3, min internal keys 1, min leaf keys 2; promote the first key of the right node.

- 2, 4, 6 → [2 4 6]. 8 → [2 4 6 8] overflow → [2 4] | [6 8], promote 6.
- 10 → [6 8 10]. 12 → overflow → [6 8] | [10 12], promote 10 → root [6 10].
- 14 → [10 12 14]. 16 → overflow → [10 12] | [14 16], promote 14 → root [6 10 14].
- 18 → [14 16 18]. 20 → overflow → [14 16] | [18 20], promote 18 → root [6 10 14 18] **overflow** (max 3).
- **Root split:** left [6 10], **push 14 up**, right [18] ⇒ height increases.

**Example 4 (slide homework):** keys F, S, Q, K, C, L, H, T, V, W, M, R

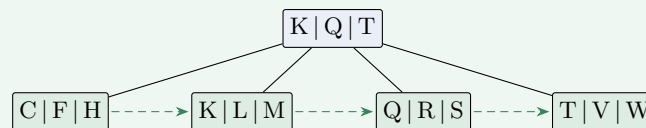
Order 3 (max 2 keys; leaf split 1 | 2 with copy-up; internal split pushes the middle up):

F → [F]; S → [F S]; Q → [F Q S] split → [F] | [Q S], root [Q]; K → [F K]; C → [C F K] split → [C] | [F K], root [F Q]; L → [F K L] split → [F] | [K L], root [F K Q] overflow → push K: root [K], internals [F], [Q]; H → [F H]; T → [Q S T] split → [Q] | [S T], internal [Q S]; V → [S T V] split → [S] | [T V], internal [Q S T] overflow → push S: root [K S], internals [Q], [T]; W → [T V W] split → [T] | [V W], internal [T V]; M → [K L M] split → [K] | [L M], internal [L Q]; R → [Q R].

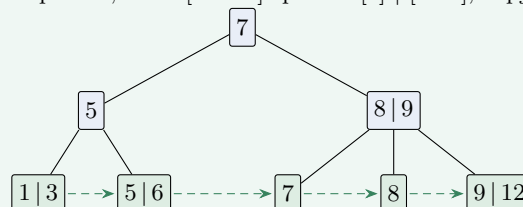


Order 4 (max 3 keys; leaf split 2 | 2 with copy-up):

F, S, Q → [F Q S]; K → [F K Q S] split → [F K] | [Q S], root [Q]; C → [C F K]; L → [C F K L] split → [C F] | [K L], root [K Q]; H → [C F H]; T → [Q S T]; V → [Q S T V] split → [Q S] | [T V], root [K Q T]; W → [T V W]; M → [K L M]; R → [Q R S].

**Example 5 (textbook keys):** B⁺-tree of order 3, insert 8, 5, 1, 7, 3, 12, 9, 6

8, 5 → [5 8]; 1 → [1] | [5 8], root [5]; 7 → [5 7 8] split → [5] | [7 8], root [5 7]; 3 → [1 3]; 12 → [7 8 12] split → [7] | [8 12], copy 8 → root [5 7 8] overflow → push 7; 9 → [8 9 12] split → [8] | [9 12], copy 9 → internal [8 9]; 6 → [5 6].



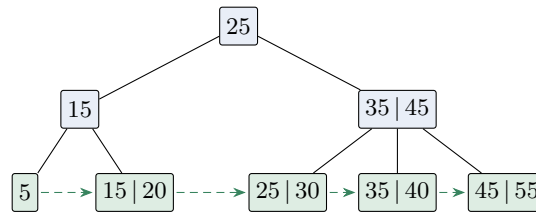
Compare with the B-tree of the same keys (§2.2): the B-tree has 2 levels with each key once; the B⁺-tree has 3 levels, and keys 5, 7, 8, 9 appear twice.

5 B⁺-Tree Deletion

Algorithm

1. Find the leaf N containing K and delete K .
2. If N still has **at least the minimum** keys $\Rightarrow$ done. Update a parent separator if needed.
3. Otherwise **underflow**:
 - **Borrow (redistribute)** a key from an immediate sibling that has more than the minimum, and update the parent separator; **or**
 - **Merge** N with a sibling and remove the separator from the parent. The parent may underflow in turn, so repeat upward.
4. If the root ends up with no keys (one child), remove it: **height** -1 .

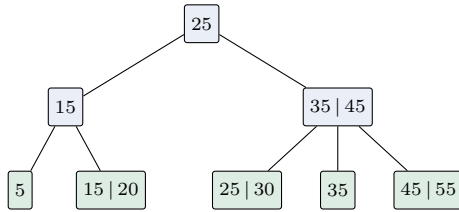
5.1 Deletion cases (slides, order $m = 3$: max 2 keys, min 1 key per non-root node)



Starting tree

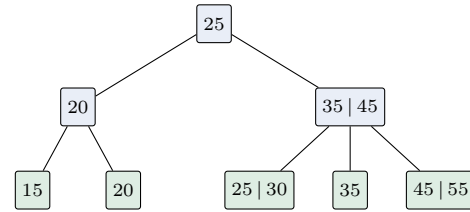
Case I(a) – Delete 40

Key only in a leaf that has more than the minimum $\Rightarrow$ simply delete.

Leaf [35 40] $\rightarrow$ [35].

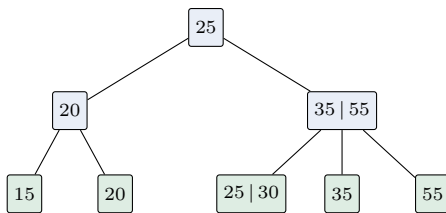
Case I(b) – Delete 5

Leaf at the minimum $\Rightarrow$ delete, borrow from the immediate sibling, update the parent.

Borrow 15 from [15 20]; parent 15 $\rightarrow$ 20.

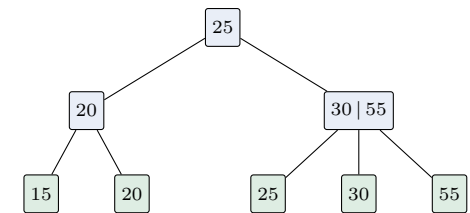
Case II(a) – Delete 45

Key in both a leaf and an internal node $\Rightarrow$ delete from the leaf, replace the internal key with its in-order successor.

Leaf [45 55] $\rightarrow$ [55]; internal [35 45] $\rightarrow$ [30 55].

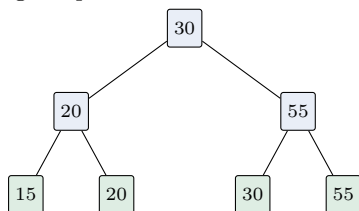
Case II(b) – Delete 35

Leaf drops below the minimum $\Rightarrow$ borrow from the sibling through the parent; the borrowed key fills the internal node.

Borrow 30 from [25 30]; internal [35 55] $\rightarrow$ [30 55].

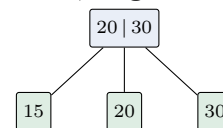
Case II(c) – Delete 25

Underflow reaches the parent $\Rightarrow$ merge with the sibling, update the grandparent with the in-order successor.

Empty leaf merges away; root 25 $\rightarrow$ 30; internal [30 55] $\rightarrow$ [55].

Case III – Delete 55

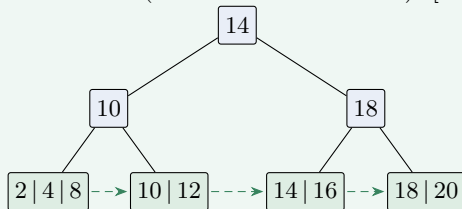
Underflow propagates to the root $\Rightarrow$ root becomes empty and is removed; **height** -1 .

Internal [55] underflows, sibling [20] can't lend $\Rightarrow$ root 30 comes down: [20 30].

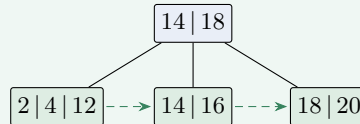
5.2 Deletion example from the construction problem (order $m = 4$; delete 6, 8, 10, 12)

Start: final tree of Example 3 (min leaf keys 2, min internal keys 1)

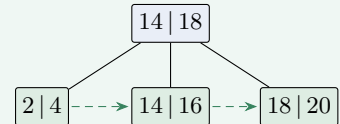
1. **Delete 6:** $[6\ 8] \rightarrow [8]$ underflow. The siblings $[2\ 4]$ and $[10\ 12]$ have exactly 2 keys (can't lend) $\Rightarrow$ **merge left:** $[2\ 4\ 8]$. Remove separator 6 $\rightarrow$ parent $[10]$.
2. **Delete 8:** $[2\ 4\ 8] \rightarrow [2\ 4]$, still valid.
3. **Delete 10:** $[10\ 12] \rightarrow [12]$ underflow. Left sibling $[2\ 4]$ has only 2 $\Rightarrow$ merge: $[2\ 4\ 12]$. Parent $[10]$ now has 0 keys $\Rightarrow$ **internal underflow**.
4. **Internal merge:** merge with sibling $[18]$, **bringing down root key 14** $\Rightarrow [14\ 18]$. Root empty $\Rightarrow$ removed, **height reduced**.
5. **Delete 12** (not shown on the slides): $[2\ 4\ 12] \rightarrow [2\ 4]$, still valid.



after deleting 6



after 10 (slide "final")

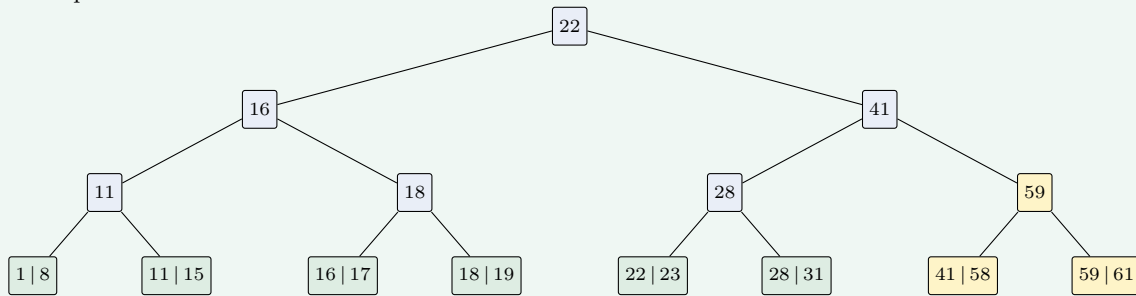


after 12 (true final)

5.3 Delete 52 (slide – result not shown there)

Solution with leaf min = 2

Leaf $[41\ 52] \rightarrow [41]$ underflows. The right sibling $[58\ 59\ 61]$ has 3 keys $>$ min $\Rightarrow$ **borrow 58** $\Rightarrow$ leaves $[41\ 58]$ and $[59\ 61]$; parent separator 58 $\rightarrow$ **59**.



Before: the right-most internal node was $[58]$ with leaves $[41\ 52]$ and $[58\ 59\ 61]$. The highlighted nodes changed. (If the leaf minimum were 1, $[41]$ would be valid and nothing else would change.)

6 B-Tree vs. B⁺-Tree

B-tree	B ⁺ -tree
Data pointers in all nodes	Data pointers only in the leaves
Each key appears once	Internal keys are repeated in the leaves
No leaf chain	Leaves linked $\Rightarrow$ fast range/sequential scans
Internal nodes store $\langle K, Pr \rangle \Rightarrow$ lower fan-out	Internal nodes store only keys + tree pointers $\Rightarrow$ higher fan-out, shallower tree
Search may stop early at an internal node	Search always ends at a leaf (uniform cost)
Split: median moves up	Leaf split: copy up; internal split: move up
Deletion from internal nodes is complex	Deletion always starts at a leaf

Capacity calculation (Elmasri example)

Given: key $V = 9$ B, block $B = 512$ B, data pointer $Pr = 7$ B, block pointer $P = 6$ B; nodes about 69% full.

B-tree order:

$$pP + (p-1)(Pr + V) \leq B \Rightarrow 6p + 16(p-1) \leq 512 \Rightarrow 22p \leq 528 \Rightarrow p = 24$$

At 69%: ≈ 16 pointers, 15 keys per node.

B⁺-tree orders:

internal: $pP + (p-1)V \leq B \Rightarrow 15p \leq 521 \Rightarrow p = 34$; leaf: $p_{leaf}(Pr+V) + P \leq B \Rightarrow 16p_{leaf} + 6 \leq 512 \Rightarrow p_{leaf} = 31$

At 69%: internal ≈ 23 pointers (22 keys), leaf ≈ 21 data pointers.

B-tree				B ⁺ -tree			
Level	Nodes	Keys	Pointers	Level	Nodes	Keys	Pointers
Root	1	15	16	Root	1	22	23
Level 1	16	240	256	Level 1	23	506	529
Level 2	256	3,840	4,096	Level 2	529	11,638	12,167
Level 3	4,096	61,440	–	Leaves	12,167	–	255,507 data ptrs
Total		65,535		Total			255,507

With the same 4 levels the B⁺-tree indexes about **4× more records**, because its internal nodes don't waste space on data pointers. **This is why DBMSs use B⁺-trees.**

7 Index Definition in SQL

```
CREATE INDEX <index-name> ON <relation-name> (<attribute-list>);
CREATE INDEX b_index ON branch(branch_name);
CREATE UNIQUE INDEX ...      -- also enforces that the search key is a candidate key
DROP INDEX <index-name>;
```

Most RDBMSs build a B⁺-tree by default (e.g. MySQL InnoDB's primary key is a clustered B⁺-tree).

8 Slide Errata

Slide	Issue → correction
Main 136, 138; idx 38–39	“Balanced <i>binary</i> search tree” → balanced multiway tree. Linked leaves are a B ⁺ -tree feature.
Main 138	“Lead nodes” → leaf nodes; the ranges “2–4 values, 3–5 children” don't match $n = 3$.
Main 147; idx 42, 49, 53	Leaf max keys = m , leaf min = $\lceil m/2 \rceil$ → the examples use leaf max $m - 1$ and leaf min $\lceil (m - 1)/2 \rceil$.
Main 153–154	“Delete 52” result not shown → see §5.3.
Main 160	“Delete 6, 8, 10, 12” – 12 never deleted → final root [14 18] → [2 4], [14 16], [18 20].

9 Quick Revision

- **B-tree:** data pointers in every node, each key once, median moves up on a split.
- **B⁺-tree:** data pointers only in the leaves, leaves linked, higher fan-out $\Rightarrow$ shallower.
- Order m : max m children, max $m - 1$ keys; internal min $\lceil m/2 \rceil$ children ($\lceil m/2 \rceil - 1$ keys); leaf min $\lceil (m - 1)/2 \rceil$ keys; root ≥ 2 children.
- Insert: always into a leaf. Overflow $\Rightarrow$ split; **leaf split copies up, internal split pushes up**. Root split $\Rightarrow$ height +1.
- Delete: always from a leaf. Underflow $\Rightarrow$ **borrow** from a sibling (update separator) else **merge** (remove separator). Empty root $\Rightarrow$ height –1.
- Internal key deleted $\Rightarrow$ replace it with the **in-order successor**.
- Search / insert / delete cost $\approx$ height = $O(\log_{\lceil m/2 \rceil} N)$.
- Capacity formulas: B-tree $pP + (p-1)(Pr+V) \leq B$; B⁺ internal $pP + (p-1)V \leq B$; B⁺ leaf $p_{leaf}(Pr+V) + P \leq B$.